Rapport de Projet de Programmation

Recherche de Chemin Optimal sous Contrainte de Fatigue

ENSAE 2025-2026

Projet encadré par Guillaume Levy

Raphaël Coic - Lucas Poignant

19 avril 2026

Table des matières

1 Introduction Générale **1**

2 Partie 1 : Chemin de temps minimal **2**

2.1 1. Le Graphe Simple : Cas sans augmentation de fatigue 2

2.2 2. Le Graphe Étendu : Prise en compte de la fatigue 2

2.3 3. Le Graphe Implicite : Optimisation structurelle 3

3 Partie 2 : Améliorations algorithmiques **5**

3.1 1. Élagage (Pruning) par Dominance de Pareto 5

3.2 2. Algorithme A* et Heuristique Admissible 7

4 Partie 3 : Extensions (Missions Multiples) **8**

4.1 1. Évaluation d'une séquence imposée de missions 8

4.2 2. Optimisation de l'ordre des missions (Cas du choix libre) 8

5 Conclusion Générale et Prédictions **9**

1 Introduction Générale

Le problème du plus court chemin est un pilier fondamental de la recherche opérationnelle et de l'algorithmique sur les graphes. Classiquement résolu par des algorithmes tels que ceux de Dijkstra, Bellman-Ford ou A*, ce problème suppose généralement que le coût de transition entre deux états (le poids d'une arête) est statique et indépendant de l'historique du parcours.

Cependant, le présent projet introduit une complexité novatrice et particulièrement réaliste : **la fatigue de l'agent**. Dans notre environnement modélisé, chaque déplacement n'a pas seulement un coût temporel lié à la distance physique (la longueur de la route), mais modifie également l'état interne de l'agent en augmentant son niveau de fatigue.

La contrainte majeure réside dans le fait que la vitesse de l'agent est inversement proportionnelle à sa fatigue. Le temps de parcours d'une arête devient donc : $temps = longueur \times fatigue_actuelle$. De plus, à l'issue de la traversée, la fatigue de l'agent s'accroît selon un coefficient propre à l'arête (lié par exemple à la difficulté du terrain).

Cette dynamique rompt la propriété de Markov sur un graphe spatial simple : le coût optimal depuis un sommet jusqu'à la destination dépend fortement du chemin emprunté pour arriver à ce sommet. Un long détour plat pourrait s'avérer globalement plus rapide qu'un raccourci exténuant.

L'objectif de notre projet est de concevoir une architecture logicielle robuste en Python permettant de résoudre ce problème. Ce rapport retrace notre progression analytique et algorithmique. Nous commencerons par l'étude de graphes simples, avant de modéliser le problème sous forme de graphe d'états (graphe étendu puis implicite). Nous détaillerons ensuite les optimisations massives apportées par l'élagage de Pareto et l'heuristique A*. Enfin, nous analyserons nos solutions apportées aux extensions du problème impliquant des missions multiples.

2 Partie 1 : Chemin de temps minimal

Cette première partie vise à établir les fondations de notre algorithme de recherche en explorant différentes représentations du problème.

2.1 1. Le Graphe Simple : Cas sans augmentation de fatigue

Intérêt de la question :
Avant de s'attaquer à la complexité de l'état de fatigue cumulatif, il est impératif de valider notre infrastructure de base. Fixer l'augmentation de fatigue à zéro ($f(e) = 0$ pour toute arête) ramène mathématiquement le multiplicateur de fatigue à une constante égale à 1. Le problème dégénère alors en une recherche de plus court chemin standard. Cette étape permet de tester notre parseur de fichiers (`Network.from_file`) et le bon fonctionnement de notre algorithme de recherche (`Graph.shortest_path`) sans le bruit des variables dynamiques.

Réalisation dans le code :
Nous avons implémenté la méthode `build_simple_graph` au sein de la classe `Network`. Cette méthode itère sur les arêtes originelles du réseau et ignore délibérément le coefficient d'augmentation de fatigue, ne conservant que la destination et la longueur physique. Dans la classe `Graph`, la méthode `shortest_path` implémente un algorithme de Dijkstra glouton qui opère sur ces nœuds simples (représentés par de simples chaînes de caractères).

Justification des choix algorithmiques et Complexité :
Pour cette version de base, l'algorithme agit comme un Dijkstra classique. À chaque itération, il recherche le sommet atteignable avec le coût minimal.

- **Complexité Spatiale :** $\mathcal{O}(V + E)$ pour stocker le graphe en mémoire sous forme de liste d'adjacence, et $\mathcal{O}(V)$ pour le dictionnaire des `distances`.
- **Complexité Temporelle :** Notre implémentation utilise une recherche de minimum linéaire dans le dictionnaire des distances : `current = min(...)`. Dans le pire des cas, cela donne une complexité de $\mathcal{O}(V^2)$. Bien qu'une file de priorité (Min-Heap) permettrait d'atteindre $\mathcal{O}(E + V \log V)$, la structure dictionnaire est suffisante et plus flexible pour nos évolutions ultérieures.

2.2 2. Le Graphe Étendu : Prise en compte de la fatigue

Intérêt de la question :
Pour modéliser la fatigue, le graphe spatial ne suffit plus. Atteindre le sommet "A" avec une fatigue de 1 et atteindre ce même sommet avec une fatigue de 15 constituent deux réalités physiques (deux *états*) totalement différentes. La formulation équivalente au problème markovien consiste à "déplier" le graphe spatial en un graphe d'états, appelé Graphe Étendu, où chaque nœud devient un tuple $(sommet, fatigue)$.

Réalisation dans le code :
La méthode `build_extended_graph(max_fatigue)` génère la totalité de l'espace d'état en amont. Elle crée une nouvelle liste d'adjacence où, pour chaque niveau de fatigue possible (de 1 à `max_fatigue`), on pré-calcule les transitions. Le coût d'une arête devient `longueur * fatigue_actuelle`, et la destination devient `(dest, fatigue_actuelle + augmentation_fatigue)`. La classe `GraphExtended` hérite de `Graph` et branche sa fonction `neighbours` sur ce méga-dictionnaire.

Justification des choix algorithmiques et Complexité :
C'est une approche *brute-force*. Le choix de pré-calculer permet d'utiliser notre algorithme `shortest_path` sans **aucune** modification interne.

— **Complexité Spatiale :** Désastreuse. Si l'on note F_{max} la limite de fatigue, le nombre de sommets devient $V \times F_{max}$ et le nombre d'arêtes $E \times F_{max}$. La complexité spatiale explose à $\mathcal{O}(V \cdot F_{max} + E \cdot F_{max})$.

— **Complexité Temporelle :** Le temps de génération du graphe est en $\mathcal{O}(V \cdot F_{max})$. Le temps d'exécution de la recherche s'élève à $\mathcal{O}((V \cdot F_{max})^2)$ avec notre extracteur de minimum.

2.3 3. Le Graphe Implicite : Optimisation structurelle

Intérêt de la question :
Face à l'explosion combinatoire de la RAM mise en évidence par l'approche étendue, une solution s'impose : l'évaluation paresseuse (*lazy evaluation*). Il est formellement inutile de générer les arêtes partant du sommet "B" avec une fatigue de 500 si l'agent ne peut matériellement jamais atteindre "B" avec ce niveau de fatigue. Le graphe implicite ne génère les voisins que lorsque l'algorithme a effectivement besoin de les visiter.

Réalisation dans le code :
La fonction `build_implicit_graph` de la classe `Network` encapsule la logique de transition dans une *closure* : la fonction locale `compute_neighbours`. La classe `GraphImplicit` reçoit cette fonction en argument. Ainsi, quand la méthode `shortest_path` demande les voisins d'un nœud (v, f), la fonction génère à la volée la liste des arêtes valides à cet instant précis. L'empreinte mémoire du graphe redevient celle du réseau originel.

Justification des choix algorithmiques et Complexité :
Ce design pattern offre une modularité exceptionnelle : l'algorithme de recherche ignore totalement la nature du graphe sous-jacent.

— **Complexité Spatiale :** Le graphe sous-jacent redevient de taille $\mathcal{O}(V + E)$. La complexité spatiale de l'algorithme est désormais bornée par le nombre strict de nœuds *visités*, que nous noterons $V_{explore}$.

— **Complexité Temporelle :** Le temps de construction du graphe est réduit à $\mathcal{O}(1)$. Le temps de recherche devient $\mathcal{O}(V_{explore}^2)$.

Résultats des Tests : Graphe Implicite (Baseline)

Environnement : Fichier `medium-smallfatigue.txt`
Temps d'exécution de la recherche : $\sim$ 2 secondes
Nœuds explorés (Dijkstra naïf, sans optimisations) : $\sim$ 4000 sommets visités

Interprétation :
Le passage au graphe implicite permet à notre algorithme de s'exécuter et de trouver le chemin optimal de manière très fluide (2 secondes). Néanmoins, en enregistrant les métriques internes de l'algorithme, on constate que le parcours en "tache d'huile" classique de Dijkstra étend environ 4000 états différents. Bien que la RAM soit sauvée par la génération à la volée, le temps de calcul brut reste proportionnel à ces 4000 explorations.

3 Partie 2 : Améliorations algorithmiques

Bien que le graphe implicite résolve le problème de l'espace mémoire, l'algorithme de base explore plus de 4000 sommets en raison des multiples chemins possibles atteignant un même point avec des fatigues variées. Cette section détaille nos méthodes de réduction drastique de l'espace de recherche (l'élagage) et de guidage de l'exploration (A*).

3.1 1. Élagage (Pruning) par Dominance de Pareto

Intérêt de la question :
En explorant le graphe, nous générons une multitude d'états pour un même sommet physique. Supposons deux chemins, C_1 et C_2, menant tous deux au sommet v.

— C_1 mène à v en 50 minutes, avec une fatigue accumulée de 3.

— C_2 mène à v en 60 minutes, avec une fatigue accumulée de 4.

Le chemin C_2 est strictement inférieur à C_1 sur nos deux critères de minimisation (temps et fatigue). Il est dominé au sens de Pareto. Continuer d'explorer les voisins depuis C_2 garantit d'obtenir des temps globaux pires que depuis C_1. L'élagage permet de "couper" ces branches mortes.

Réalisation dans le code :
Nous avons codé la méthode `_is_pareto_dominated(self, node)`. Avant d'étendre un nœud courant extrait de notre frontière d'exploration, nous vérifions si, pour le même sommet physique, le dictionnaire des distances contient déjà un état atteignable avec un temps $\leq$ et une fatigue $\leq$ (l'un des deux étant strict). Si oui, la boucle passe au nœud suivant sans explorer ses voisins.

Justification des choix algorithmiques et Complexité :
Ce test induit un surcoût mineur de calcul à chaque itération (parcours de $\mathcal{O}(V_{explore})$ états). Cependant, son impact global réduit la taille de $V_{explore}$ de manière drastique, comme le prouvent nos tests.

Résultats des Tests : Élagage de Pareto

Environnement : Fichier `medium-smallfatigue.txt`
Nœuds étendus (algorithme naïf) : $\sim$ 4000 sommets
Nœuds étendus (avec Pruning de Pareto) : 1300 sommets

Interprétation :
L'activation de l'élagage de Pareto supprime net environ **67.5% de l'espace d'exploration**. Cette diminution spectaculaire prouve mathématiquement que la grande majorité (plus des deux tiers) des chemins explorés par l'algorithme initial représentaient des détours inutiles et épuisants. En les coupant à la racine, le programme économise considérablement en ressources de calcul.

3.2 2. Algorithme A* et Heuristique Admissible

Intérêt de la question :

Même avec l'élagage, Dijkstra (avec pruning) continue de chercher dans toutes les directions. L'algorithme A* intègre une *heuristique* $h(n)$ qui estime le coût restant depuis un nœud n jusqu'à la cible. La priorité d'exploration devient $f(n) = g(n) + h(n)$ où $g(n)$ est le coût déjà accumulé. Pour garantir que A* trouve le chemin **optimal**, l'heuristique doit être **admissible**, c'est-à-dire qu'elle ne doit **jamais** surestimer le coût réel restant.

Réalisation dans le code :

L'intégration de A* s'opère dans `shortest_path` via le changement de clé de tri : `key=lambda x: self.distances[x] + self.heuristic(x, end)`. Notre fonction heuristique procède ainsi :

1. Lors du premier appel vers une cible v_t, la méthode `_compute_estimated_distances` effectue un Dijkstra "à l'envers" depuis v_t sur le graphe de base (en ignorant la fatigue). Cela nous donne la distance physique minimale pure, notée $d_{base}(v, v_t)$.
2. L'heuristique pour un nœud (v, f) devient : $h(v, f) = d_{base}(v, v_t) \times f$.

Justification de l'admissibilité (Preuve mathématique) :

Soit un état (v, f). Pour atteindre v_t, l'agent devra emprunter un ensemble d'arêtes $E_{restant}$. Le vrai coût restant est $C_{reel} = \sum_{e \in E_{restant}} l(e) \times f_e$. Or, la fatigue est croissante de manière monotone ($f(e) \geq 0$), donc pour toute arête future, $f_e \geq f$. Ainsi, $C_{reel} \geq \sum_{e \in E_{restant}} l(e) \times f = f \times \sum l(e)$. Par définition du plus court chemin de base, la somme réelle des longueurs $\sum l(e)$ est supérieure ou égale au chemin physique optimal $d_{base}(v, v_t)$. On a donc formellement : $C_{reel} \geq f \times d_{base}(v, v_t) = h(v, f)$. L'heuristique est donc un **minorant strict** du coût réel ; elle est parfaitement admissible.

Résultats des Tests : Algorithme A*

Environnement : Fichier `medium-smallfatigue.txt`
Nœuds étendus (avec Pruning uniquement) : 1300 sommets
Nœuds étendus (avec Pruning + Heuristique A*) : 475 sommets

Interprétation :

L'ajout de l'heuristique divise à nouveau par près de 3 le nombre d'états visités. L'algorithme ne s'éparpille plus en cercles concentriques : la fonction de coût "tire" l'exploration puissamment vers la destination. Au total, la combinaison Pareto + A* a permis de **réduire l'espace de recherche de près de 88%** (de ∼4000 à 475 nœuds) par rapport à un parcours de graphe implicite naïf.

4 Partie 3 : Extensions (Missions Multiples)

Cette section documente notre travail sur la variante des "Missions Multiples" pour un seul agent, pour laquelle nous avons créé la classe spécialisée `MultipleMissionsGraph`.

4.1 1. Évaluation d'une séquence imposée de missions

Intérêt de la question :
Lorsque l'agent doit accomplir les missions $M_1(v_{s1} \to v_{t1})$, puis $M_2(v_{s2} \to v_{t2})$, le problème principal est le transfert de la fatigue. L'agent n'est pas réinitialisé entre deux missions : la fatigue avec laquelle il termine M_1 sera celle avec laquelle il entamera le transit de v_{t1} vers v_{s2}, puis la mission M_2.

Réalisation dans le code :
La méthode `shortest_path_multiple_missions(missions)` réalise un chaînage exact. Le booléen `multiple_missions=True` de notre A* ordonne à l'algorithme de ne pas s'arrêter au premier chemin atteignant la cible, mais de collecter l'ensemble de la frontière de Pareto des états d'arrivée (v_{t1}, f_i). Le code relance ensuite l'algorithme vers la cible suivante pour chacun de ces états finaux, en propageant scrupuleusement la distance cumulée et la fatigue acquise.

Justification :
Ce chaînage de frontières de Pareto garantit que nous ne perdons pas le chemin optimal global (qui pourrait nécessiter d'arriver à la fin de M_1 avec un peu plus de temps mais moins de fatigue, afin de réussir le long trajet de M_2).

4.2 2. Optimisation de l'ordre des missions (Cas du choix libre)

Intérêt et Réalisation :
Lorsqu'on peut choisir l'ordre d'un ensemble de k missions, le problème s'apparente au TSP (Voyageur de Commerce). Nous avons proposé deux approches :

1. **Une heuristique rapide (`best_missions_order_brute`) :** Elle pré-calcule les distances de transit minimales (en ignorant la fatigue de liaison) et cherche la permutation de missions minimisant cette distance pure.
2. **Un solveur exact (`best_shortest_path_multiple_missions`) :** Il itère sur l'ensemble des $k!$ permutations de l'ordre des missions, et soumet chacune d'elle à l'algorithme de chaînage exact décrit précédemment.

Complexité :
La méthode exacte a une complexité factorielle $\mathcal{O}(k! \times C_{chainage})$. Elle n'est utilisable que pour de petites valeurs de k (typiquement $k \leq 6$ ou 8). C'est le comportement attendu pour un problème combinatoire intrinsèquement difficile.

5 Conclusion Générale et Prédictions

Ce projet met brillamment en évidence la fracture entre la théorie markovienne simple des graphes et la réalité modélisée par des agents à l'état dynamique.

Nos expériences démontrent une tendance sans appel : l'approche naïve par construction *ex-ante* du graphe d'états (le Graphe Étendu) est rapidement vaincue par l'explosion combinatoire. L'empreinte mémoire devient le goulot d'étranglement majeur bien avant le temps de calcul du processeur.

La création du **Graphe Implicite** a permis d'évaluer paresseusement les nœuds, supprimant la barrière limitante de la mémoire. Par-dessus cela, l'orchestration de nos optimisations algorithmiques a livré des résultats impressionnants. Les données chronométriques parlent d'elles-mêmes : sur le réseau moyen, l'algorithme brut explorait près de **4000 sommets** ; en activant l'élagage restrictif de Pareto, nous avons ramené ce chiffre à **1300** ; enfin, la canalisation de la recherche par notre heuristique A* admissible a achevé de réduire cet espace à **seulement 475 nœuds**, assurant des temps de réponse d'environ 2 secondes. Il s'agit d'une optimisation globale d'un facteur proche de 10, garantissant une redoutable efficacité d'exécution.

Perspectives d'amélioration : Entrelacement des missions En guise d'ouverture sur la gestion des missions multiples, une évolution majeure consisterait à relâcher la contrainte de stricte séquentialité. Actuellement, l'agent doit impérativement achever une mission avant d'en entamer une nouvelle. On pourrait imaginer un modèle où les tâches s'entrelacent : l'agent aurait la possibilité de commencer une seconde mission alors qu'il est déjà en transit pour en terminer une première. Dans ce paradigme, les points de départ et d'arrivée ne sont plus des blocs isolés, mais deviennent un ensemble de "checkpoints" globaux à valider sous de simples contraintes d'antériorité (le départ A doit être visité avant l'arrivée A). Cette flexibilité accrue, qui rapproche notre modèle du problème classique de ramassage et livraison (Pickup and Delivery Problem), permettrait d'explorer un espace de solutions bien plus vaste et de réduire encore plus drastiquement la longueur et le temps total du parcours de l'agent.

Fin du rapport.